Advanced

Course: Rust Basics

Author: Aleksandr Kugushev

Type aliases

```
type Point = (u8, u8);
let p: Point = (41, 68);


struct MyStruct(u32);

use MyStruct as UseAlias;
type TypeAlias = MyStruct;

let _ = UseAlias(5); // OK
let _ = TypeAlias(5); // Doesn't work
```

Error Handling

Recoverable

```
fn recoverable(input: i32) -> Result<i32, &'static
str>{
    if input < 0 {
        return Result::Err("Input is less than zero");
    }

    Result::Ok(input + 42)
}
```

Unrecoverable

```
fn unrecoverable(input: i32) -> i32 {
    if input < 0 {
        panic!("Input is less than zero");
    }

    input + 42
}
```

panic!()

```
fn unrecoverable(input: i32) -> i32 {
    if input < 0 {
        panic!("Input is less than zero");
    }

    input + 42
}
```

Panic strategies

- Unwind
- Abort
- Panic Handler

Unwind

```
fn main() {
    let str1 = MyStruct { value: 1 };
    function();
}

fn function() {
    let str2 = MyStruct { value: 2 };
    panic!("Oops")
}

struct MyStruct {
    value: i32,
}

impl Drop for MyStruct {
    fn drop(&mut self) { println!("Drop {}", self.value) }
}
```

Unwind

```
thread 'main' panicked at 'Oops', src\main.rs:359:5
stack backtrace:
  0: std::panicking::begin_panic_handler
             at /rustc/fc594f15669680fa70d255faec3ca3fb507c3405/library\std\src\panicking.rs:575
  1: core::panicking::panic_fmt
             at /rustc/fc594f15669680fa70d255faec3ca3fb507c3405/library\core\src\panicking.rs:64
  2: hello_bin::function
             at .\src\main.rs:359
  3: hello_bin::main
             at .\src\main.rs:351
  4: core::ops::function::FnOnce::call_once<void (*)(),tuple$<> >
             at /rustc/fc594f15669680fa70d255faec3ca3fb507c3405\library\core\src\ops\function.rs:507
note: Some details are omitted, run with `RUST_BACKTRACE=full` for a verbose backtrace.
Drop 2
Drop 1
error: process didn't exit successfully: `target\debug\hello-bin.exe` (exit code: 101)
```

RUST_BACKTRACE=full

```
0:   0x7ff736b38782 - std::backtrace_rs::backtrace::dbghelp::trace
                 at /rustc/fc594f15669680fa70d255faec3ca3fb507c3405/library\std\src\..\..\backtrace\src\backtrace\dbghelp.rs:98
 1:   0x7ff736b38782 - std::backtrace_rs::backtrace::trace_unsynchronized
                 at /rustc/fc594f15669680fa70d255faec3ca3fb507c3405/library\std\src\..\..\backtrace\src\backtrace\mod.rs:66
 2:   0x7ff736b38782 - std::sys_common::backtrace::_print_fmt
                 at /rustc/fc594f15669680fa70d255faec3ca3fb507c3405/library\std\src\sys_common\backtrace.rs:65
 3:   0x7ff736b38782 - std::sys_common::backtrace::_print::impl$0::fmt
                 at /rustc/fc594f15669680fa70d255faec3ca3fb507c3405/library\std\src\sys_common\backtrace.rs:44
 4:   0x7ff736b4783b - core::fmt::write
                 at /rustc/fc594f15669680fa70d255faec3ca3fb507c3405/library\core\src\fmt\mod.rs:1208
 5:   0x7ff736b3683a - std::io::Write::write_fmt<std::sys::windows::stdio::Stderr>
                 at /rustc/fc594f15669680fa70d255faec3ca3fb507c3405/library\std\src\io\mod.rs:1682
 6:   0x7ff736b384cb - std::sys_common::backtrace::_print
                 at /rustc/fc594f15669680fa70d255faec3ca3fb507c3405/library\std\src\sys_common\backtrace.rs:47
 7:   0x7ff736b384cb - std::sys_common::backtrace::print
                 at /rustc/fc594f15669680fa70d255faec3ca3fb507c3405/library\std\src\sys_common\backtrace.rs:34
 8:   0x7ff736b3a6c9 - std::panicking::default_hook::closure$1
                 at /rustc/fc594f15669680fa70d255faec3ca3fb507c3405/library\std\src\panicking.rs:267
 9:   0x7ff736b3a34b - std::panicking::default_hook
                 at /rustc/fc594f15669680fa70d255faec3ca3fb507c3405/library\std\src\panicking.rs:286
10:   0x7ff736b3af61 - std::panicking::rust_panic_with_hook
                 at /rustc/fc594f15669680fa70d255faec3ca3fb507c3405/library\std\src\panicking.rs:688
11:   0x7ff736b3acab - std::panicking::begin_panic_handler::closure$0
                 at /rustc/fc594f15669680fa70d255faec3ca3fb507c3405/library\std\src\panicking.rs:577
12:   0x7ff736b3919f - std::sys_common::backtrace::__rust_end_short_backtrace<std::panicking::begin_panic_handler::closure_env$0,never$>
                 at /rustc/fc594f15669680fa70d255faec3ca3fb507c3405/library\std\src\sys_common\backtrace.rs:137
13:   0x7ff736b3a9a0 - std::panicking::begin_panic_handler
                 at /rustc/fc594f15669680fa70d255faec3ca3fb507c3405/library\std\src\panicking.rs:575
14:   0x7ff736b4d615 - core::panicking::panic_fmt
                 at /rustc/fc594f15669680fa70d255faec3ca3fb507c3405/library\core\src\panicking.rs:64
15:   0x7ff736b310a3 - hello_bin::function
                 at C:\Projects\learn\rust\playground\hello-world\hello-bin\src\main.rs:352
16:   0x7ff736b3101e - hello_bin::main
                 at C:\Projects\learn\rust\playground\hello-world\hello-bin\src\main.rs:347
17:   0x7ff736b3129b - core::ops::function::FnOnce::call_once<void (*)(),tuple$<> >
                 at /rustc/fc594f15669680fa70d255faec3ca3fb507c3405\library\core\src\ops\function.rs:507
18:   0x7ff736b3138e - std::sys_common::backtrace::__rust_begin_short_backtrace<void (*)(),tuple$<> >
                 at /rustc/fc594f15669680fa70d255faec3ca3fb507c3405\library\std\src\sys_common\backtrace.rs:121
19:   0x7ff736b3138e - std::sys_common::backtrace::__rust_begin_short_backtrace<void (*)(),tuple$<> >
                 at /rustc/fc594f15669680fa70d255faec3ca3fb507c3405\library\std\src\sys_common\backtrace.rs:121
20:   0x7ff736b31401 - std::rt::lang_start::closure$0<tuple$<> >
                 at /rustc/fc594f15669680fa70d255faec3ca3fb507c3405\library\std\src\rt.rs:166
21:   0x7ff736b3399e - core::ops::function::impls::impl$2::call_once
                 at /rustc/fc594f15669680fa70d255faec3ca3fb507c3405/library\core\src\ops\function.rs:606
22:   0x7ff736b3399e - std::panicking::try::do_call
                 at /rustc/fc594f15669680fa70d255faec3ca3fb507c3405/library\std\src\panicking.rs:483
23:   0x7ff736b3399e - std::panicking::try
                 at /rustc/fc594f15669680fa70d255faec3ca3fb507c3405/library\std\src\panicking.rs:447
```

Abort

Cargo.toml

- [profile.dev]
  panic = 'abort'

Abort

```
thread 'main' panicked at 'Oops', src\main.rs:352:5
stack backtrace:
   0: std::panicking::begin_panic_handler
             at /rustc/fc594f15669680fa70d255faec3ca3fb507c3405/library\std\src\panicking.rs:575
   1: core::panicking::panic_fmt
             at /rustc/fc594f15669680fa70d255faec3ca3fb507c3405/library\core\src\panicking.rs:64
   2: hello_bin::function
             at .\src\main.rs:352
   3: hello_bin::main
             at .\src\main.rs:347
   4: core::ops::function::FnOnce::call_once<void (*)(),tuple$<> >
             at /rustc/fc594f15669680fa70d255faec3ca3fb507c3405\library\core\src\ops\function.rs:507
note: Some details are omitted, run with `RUST_BACKTRACE=full` for a verbose backtrace.
error: process didn't exit successfully: `target\debug\hello-bin.exe` (exit code: 0xc0000409, STATUS_STACK_BUFFER_OVERRUN)
```

Panic Handler

```
#![no_std]


#[panic_handler]
fn panic(info: &PanicInfo) -> ! {
    let mut host_stderr = HStderr::new();

    writeln!(host_stderr, "{}", info).ok();

    loop {}
}
```

Catch Unwind

```
fn main() {
    let result = panic::catch_unwind(|| {
        let str1 = MyStruct { value: 1 };
        function();
    });

    match result {
        Ok(_) => {}
        Err(panic) => {
            let msg = panic.downcast::<&str>().unwrap();
            println!("Help: {}", msg);
        }
    }
}
```

Catch Unwind

Drop 2

Drop 1

Help: Oops

Result<T,E>

```rust
pub enum Result<T, E> {
    Ok(T),
    Err(E),
}


fn recoverable(input: i32) -> Result<i32, &'static str> {
    if input < 0 {
        return Result::Err("Input is less than zero");
    }

    Result::Ok(input + 42)
}
```

Result<(),...>

```
fn procedure(input: i32) -> Result<(), &'static str> {
    if input < 0 {
        return Result::Err("Input is less than zero");
    }

    // do something
    Result::Ok(())
}
```

How to deal with Result

```
let result = recoverable(-1);
match result {
    Ok(integer) => {}
    Err(error) => {}
}
if result.is_ok() {
    let value = result.unwrap();
}
let value = result.expect("Error"); // panic `Input is less than zero`
```

Result propagation

```
fn call() -> Result<(), &'static str> {
    let result = recoverable(-1);
    match result {
        Ok(value) => {
            println!("{}", value);
            Ok(())
        }
        Err(msg) => { Err(msg) }
    }
}
```

Winking Elvis operator ?;

```
pub fn call() -> Result<(), &'static str>{
    let result = recoverable(-1);
    let value: i32 = result?;
    println!("{}", value);
    return Ok(());
}
```

Winking Elvis operator ?;

```
pub fn call_mult() -> Result<(), &'static str> {
    let result1: i32 = recoverable(1)?;
    let result2: i32 = recoverable(2)?;
    let result3: i32 = recoverable(3)?;
    let result4: i32 = recoverable(4)?;
    let result5: i32 = recoverable(5)?;
    println!("{}", recoverable(-1)?);
    Ok(())
}
```

Winking Elvis operator ?;

```
pub fn call() -> Result<(), i32> {
    let result = recoverable(-1);
    let value: i32 = result?;
    println!("{}", value);
    return Ok(());
}
```

the trait `From<&str>` is not implemented for `i32`

Type coercions and cast

Coercion

- **Implicit**
- &mut T to &T
- ‘a to ‘static
- &T or &mut T to &U if T implements Deref<Target = U>
- &mut T to &mut U if T implements DerefMut<Target = U>
- Function item types to fn pointers
- Non capturing closures to fn pointers

Cast

- **Explicit**
- *as* operator

```
let float: f32 = 0.1;
let double: f64 = float as f64;
```

Cast

- Numeric Cast
  - let float: f32 = 0.1;
    let double: f64 = float as f64;
- Enum Cast
  - enum Color {
        Red = 0,
        Green = 1,
        Blue = 2
    }
  - let color = Color::*Red*;
    let num: i32 = color as i32;

From/Into traits

```
struct ErrorCode(i32);
impl From<i32> for ErrorCode {
    fn from(value: i32) -> Self {
        ErrorCode(value)
    }
}
impl Into<i32> for ErrorCode {
    fn into(self) -> i32 {
        self.0
    }
}
let error: ErrorCode = ErrorCode::from(42);
let num: i32 = error.into();
```

From/Into use case

```
fn do_somethings() -> Result<(), ErrorCode>{
    do_another()?;
    Ok(())
}

fn do_another() -> Result<(), i32> {
    Err(42)
}
```

Const

```
const NUM1: u32 = 123;
const NUM2: u32 = 42;

const ARRAY: [u32; 2] = [NUM1, NUM2];

const STRING: &'static str = "my string";

struct JustAStruct<'a> {
    numbers: [u32; 2],
    string: &'a str,
}

const MY_CONST_STRUCT: JustAStruct<'static> = JustAStruct {
    numbers: ARRAY,
    string: STRING,
};
```

Drop for constants

```
impl Drop for JustAStruct<'_> {
    fn drop(&mut self) { println!("Drop const 0_o") }
}
let instance0 = MY_CONST_STRUCT;
let instance1 = MY_CONST_STRUCT;
```

```
> Drop const 0_o
> Drop const 0_o
```

Constants are always inlined

```
const MY_CONST_STRUCT:
JustAStruct<'static> = JustAStruct {
    numbers: ARRAY,
    string: STRING,
};
```

```
let instance0 = MY_CONST_STRUCT;
let instance1 = MY_CONST_STRUCT;
```

```
let instance0 = JustAStruct {
    numbers: ARRAY,
    string: STRING,
};
let instance1 = JustAStruct {
    numbers: ARRAY,
    string: STRING,
};
```

Static

```
static MY_STATIC_STRUCT: JustAStruct<'static> = JustAStruct {
    numbers: ARRAY,
    string: STRING,
};
```

```
let instance = MY_STATIC_STRUCT;
let reference = &MY_STATIC_STRUCT;
let mut_reference = &mut MY_STATIC_STRUCT;
```

cannot move out of static item `MY_STATIC_STRUCT`

cannot borrow immutable static item `MY_STATIC_STRUCT` as mutable

Static mut

```
static mut MY_STATIC_STRUCT: JustAStruct<'static> = JustAStruct {
    numbers: ARRAY,
    string: STRING,
};
```

use of mutable static is unsafe and requires unsafe function or block

```
static INTERIOR_MUT: RefCell<JustAStruct<'static>> =
RefCell::new(JustAStruct {
    numbers: ARRAY,
    string: STRING,
});
```

`RefCell<JustAStruct<'static>>` cannot be shared between threads safely

References to static

```
static S: i32 = 25;
const C: &i32 = &S;
```

Associated constants

```
struct Struct;
impl Struct {
    const KEY: i32 = 42 ;
}
trait ConstantId {
    const ID: i32;
}
impl ConstantId for Struct {
    const ID: i32 = 1;
}
```

```
    assert_eq!(1, Struct::KEY);
    assert_eq!(1, Struct::ID);
```

Associated constants: default value

```
trait ConstantIdDefault {
    const ID: i32 = 1;
}
struct Struct;
struct OtherStruct;
impl ConstantIdDefault for Struct {}
impl ConstantIdDefault for OtherStruct {
    const ID: i32 = 5;
}
...
    assert_eq!(1, Struct::ID);
    assert_eq!(5, OtherStruct::ID);
```

Const functions

```
const fn const_function() -> JustAStruct<'static>{
    JustAStruct {
        numbers: ARRAY,
        string: STRING,
    }
}
const MY_CONST_STRUCT: JustAStruct<'static> =
const_function();
static MY_STATIC_STRUCT: JustAStruct<'static> =
const_function();
```

Executing code in compile time

aka const functions and const blocks

Associated constants functions

```
struct Struct;
struct GenericStruct<const ID: i32>;

impl Struct {
    // Definition not immediately evaluated
    const PANIC: () = panic!("compile-time panic");
}

impl<const ID: i32> GenericStruct<ID> {
    // Definition not immediately evaluated
    const NON_ZERO: () = if ID == 0 {
        panic!("contradiction")
    };
}
```

```
fn main() {
    // Referencing Struct::PANIC causes compilation error
    let _ = Struct::PANIC;
    // Fine, ID is not 0
    let _ = GenericStruct::<1>::NON_ZERO;
    // Compilation error from evaluating NON_ZERO with ID=0
    let _ = GenericStruct::<0>::NON_ZERO;
}
```

const blocks

```
fn foosize<T>() -> usize {
    const {
        std::mem::size_of::<T>() + 1
    }
}
```

```
fn foosize<T>() -> usize {
    {
        struct Const<T>(T);
        impl<T> Const<T> {
            const CONST: usize =
std::mem::size_of::<T>() + 1;
        }
        Const::<T>::CONST
    }
}
```

Modules

Modules in file (main.rs)

```
struct MyStruct0;
mod level1 {
    struct MyStruct1;
    mod level2 {
        struct MyStruct2;
    }
}
fn main() {
    let str0 = MyStruct0;
    let str2 = crate::level1::level2::MyStruct2;
}
```

```
error[E0603]: module `level2` is private
```

Visibility

```
struct MyStruct0;
mod level1 {
    struct MyStruct1;
    fn test(){
        let parent_str = crate::MyStruct0;
        let child_str = crate::level2::MyStruct2;
    }

    mod level2 {
        struct MyStruct2;
    }
}
```

error[E0603]: unit struct `MyStruct2` is private

Visibility

- pub
- pub(crate)
- pub(super)
- pub(self)
- pub(in *path*)

Visibility for everything

```
pub mod module {
    pub struct MyStruct1{
        pub value: i32
    }

    pub fn function(){}
}
```

pub

```
mod level1 {
    fn test() {
        let child_str = crate::level1::level2::MyStruct2;
    }
    mod level2 {
        pub struct MyStruct2;
    }
}
fn main() {
    let str2 = crate::level1::level2::MyStruct2;
}
```

error[E0603]: module `level2` is private

pub(crate)

- Visible within the current crate
- Most used

pub(super)

```
mod level1 {
    fn test() {
        let child_str = crate::level1::level2::MyStruct2;
    }
    pub(crate) mod level2 {
        pub(super) struct MyStruct2;
    }
}
fn main() {
    let str2 = crate::level1::level2::MyStruct2;
}
```

error[E0603]: unit struct `MyStruct2` is private

pub(self)

- Private
- Default attribute

pub(in *path*)

```
pub(crate) mod level1 {
    fn test() {
        let child_str = crate::level1::level2::level3::level4::MyStruct4; unit struct `MyStruct4` is private
    }

    pub(crate) mod level2 {
        fn test() {
            let child_str = crate::level1::level2::level3::level4::MyStruct4;
        }

        pub(crate) mod level3 {
            fn test() {
                let child_str = crate::level1::level2::level3::level4::MyStruct4;
            }

            pub(crate) mod level4 {
                pub(in crate::level1::level2) struct MyStruct4;
            }
        }
    }
}
```

Absolute and relative path

```
pub(crate) mod level1 {
    fn test() {
        let child_str = level2::level3::level4::MyStruct4;
    }

    pub(crate) mod level2 {
        fn test() {
            let child_str = level3::level4::MyStruct4;
        }

        pub(crate) mod level3 {
            fn test() {
                let child_str = level4::MyStruct4;
            }

            pub(crate) mod level4 {
                pub(in crate::level1::level2) struct
MyStruct4;
            }
        }
    }
}
```

```
pub(crate) mod level1 {
    fn test() {
        let child_str = crate::level1::level2::level3::level4::MyStruct4;
    }

    pub(crate) mod level2 {
        fn test() {
            let child_str = crate::level1::level2::level3::level4::MyStruct4;
        }

        pub(crate) mod level3 {
            fn test() {
                let child_str =
crate::level1::level2::level3::level4::MyStruct4;
            }

            pub(crate) mod level4 {
                pub(in crate::level1::level2) struct MyStruct4;
            }
        }
    }
}
```

use

```
use crate::level1::level2::level3::level4::MyStruct1;

use crate::level1::level2::level3::level4::MyStruct2 as
FooBar;

use crate::level1::level2::level3::level4::{MyStruct3,
MyStruct4};

use crate::level1::level2::level3::level4::*;
```

```
pub(crate) mod level1 {
    pub(crate) mod level2 {
        pub(crate) mod level3 {
            pub(crate) mod level4 {
                pub(crate) struct
MyStruct1;
                pub(crate) struct
MyStruct2;
                pub(crate) struct
MyStruct3;
                pub(crate) struct
MyStruct4;
            }
        }
    }
}
```

Default extern: std

```
use std::prelude::*;


pub use crate::marker::{Send, Sized, Sync, Unpin};
pub use crate::ops::{Drop, Fn, FnMut, FnOnce};
pub use crate::mem::drop;
pub use crate::convert::{AsMut, AsRef, From, Into};
pub use crate::iter::{DoubleEndedIterator, ExactSizeIterator};
pub use crate::iter::{Extend, IntoIterator, Iterator};
pub use crate::option::Option::{self, None, Some};
pub use crate::result::Result::{self, Err, Ok};
pub use core::prelude::v1::{
    assert, cfg, column, compile_error, concat, concat_idents, env, file, format_args,
    format_args_nl, include, include_bytes, include_str, line, log_syntax, module_path, option_env,
    stringify, trace_macros, Clone, Copy, Debug, Default, Eq, Hash, Ord, PartialEq, PartialOrd,
};
pub use core::prelude::v1::concat_bytes;
pub use core::prelude::v1::{RustcDecodable, RustcEncodable};
pub use core::prelude::v1::alloc_error_handler;
pub use core::prelude::v1::{bench, derive, global_allocator, test, test_case};
pub use core::prelude::v1::derive_const;
pub use core::prelude::v1::cfg_accessible;
pub use core::prelude::v1::cfg_eval;
pub use core::prelude::v1::type_ascribe;
pub use crate::borrow::ToOwned;
pub use crate::boxed::Box;
pub use crate::string::{String, ToString};
pub use crate::vec::Vec;
```

Files structure

- Cargo.toml
- Cargo.lock
- src
  - main.rs
  - features.rs
  - magic.rs

- main.rs

```
mod features;
mod magic;


fn main() {
    let feature =
crate::features::MyFeature;
}
```

Folders

- Cargo.toml
- Cargo.lock
- src
  - sub_folder
    - sub_features.rs
  - main.rs
  - features.rs
  - magic.rs

Folders

- Cargo.toml
- Cargo.lock
- src
  - sub_folder.rs
  - sub_folder
    - sub_features.rs
  - main.rs
  - features.rs
  - magic.rs

- sub_folder.rs

```
mod sub_feature;
```

- main.rs

```
mod sub_folder;

use sub_folder::sub_feature::*;
fn main() {
    let feature = MySubFeature;
}
```

Folders

- Cargo.toml
- Cargo.lock
- src
  - sub_folder.rs
  - sub_folder
    - sub_feature.rs
  - main.rs
  - features.rs
  - magic.rs

- sub_folder.rs

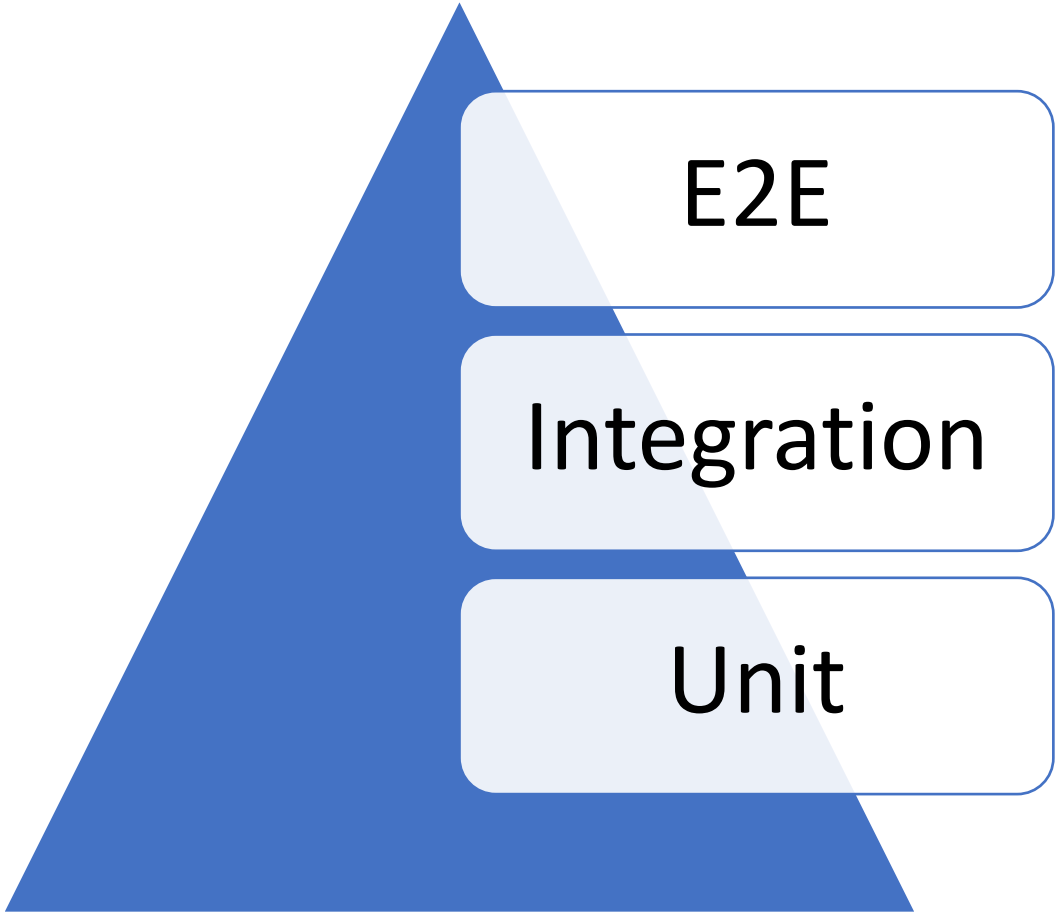
```
pub(crate) mod sub_feature;
```

- main.rs

```
mod sub_folder;

use sub_folder::sub_feature::*;
fn main() {
    let feature = MySubFeature;
}
```

Tests



Why should we need to test only public API?

```
mod my_module {
    pub fn public_api(){
        private_api();
    }

    fn private_api(){

    }
}
```

Benefits of autotests

- Regression
- … that’s all folks!

Unit Tests in Rust

- my_module.rs

```rust
pub fn summ(left: i32, right: i32) -> i32 {
    left + right
}

#[cfg(test)]
mod tests{
    use crate::my_module::summ;

    #[test]
    fn summ_trivial(){
        let result = summ(1, 2);
        assert_eq!(3, result)
    }
}
```

> cargo test

running 1 test

test my_module::tests::summ_trivial ... ok

test result: ok. 1 passed; 0 failed; 0 ignored;
0 measured; 0 filtered out; finished in 0.00s

Check panic

```rust
pub fn panic_on_42(num: i32) {
    if num == 42 { panic!("AAA!") }
}


#[cfg(test)]
mod tests {
    use crate::my_module::*;

    #[test]
    #[should_panic]
    fn panic_on_42_check_panic() {
        panic_on_42(42);
    }

}
```

Result in tests

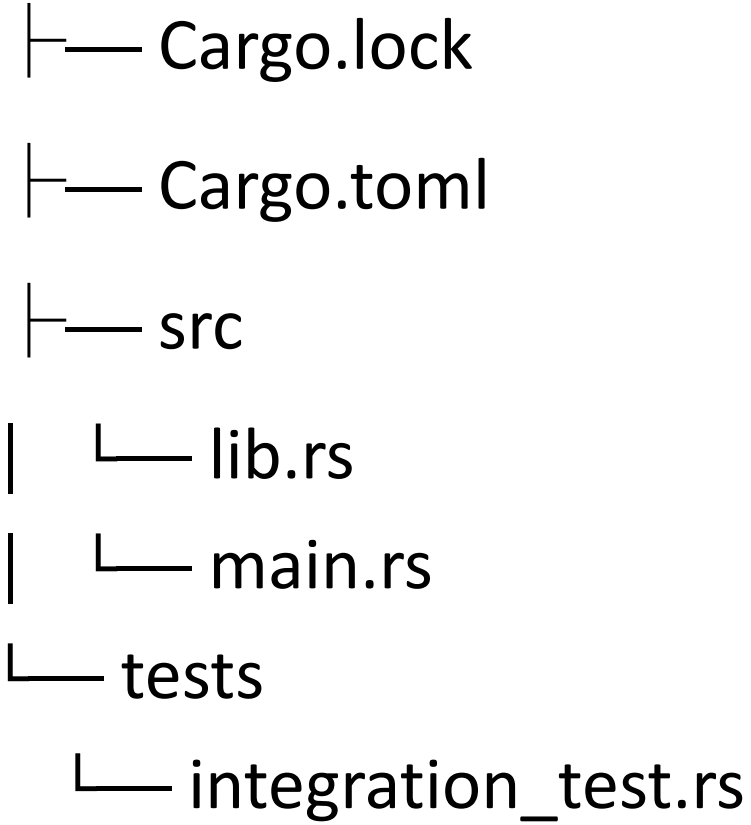
```
pub fn error_on_42(num: i32) -> Result<i32, String> {
    if num == 42
    { Err(String::from("AAA!")) } else
    { Ok(num) }
}
#[cfg(test)]
mod tests {
    use crate::my_module::*;

    #[test]
    fn error_on_42_valid_cases() -> Result<(), String> {
        error_on_42(41)?;
        error_on_42(43)?;
        Ok(())
    }
}
```

Integration tests

```
├── Cargo.lock
├── Cargo.toml
├── src
│   └── lib.rs
└── tests
    └── integration_test.rs
```

Integration tests for Console Apps



- Cargo.toml

```
[[bin]]
name = "my_app"
path = "src/main.rs"

[lib]
name = "my_app"
path = "src/lib.rs"
```

How to test println!(“…")?

- lib.rs

```
pub fn run(stdout: &mut
dyn Write){
    ...
    writeln!(stdout, "Hello");
    ...
}
```

- main.rs

```
run(&mut io::stdout())
```

- test.rs

```
let mut buffer: Vec<u8> = Vec::new();
run(&mut buffer);
let actual =
String::from_utf8(buffer).unwrap();
```

Homework

- Rustlings:
  - error_handling
- Courseware
  - Refactoring:
    - Use modules
    - Use ?; operator
  - Add tests
    - Add unit tests for core functionality
    - Add integrations tests for 2-3 use cases